

KARNATAKA RESIDENTIAL EDUCATIONAL INSTITUTIONS SOCIETY

ICT Class 10 - Summative Assessment 2

Summative Assessment (SA-2) - Answer Key

Class: Class 10

Assessment Type: Summative Assessment (SA-2)

Total Marks: 40

Duration: 2 hours

Topics Covered: Ch 1-6: Networking, HTML/CSS, Python Programming, SQL Database
Management

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Part A: MCQ (10 x 1 = 10 marks)

- Answer:** b - [1, 2, 3, 4]
 - Explanation:** Lists are mutable. `y = x` creates a reference, not a copy. Modifying `y` also modifies `x`.
- Answer:** b - ALTER TABLE ... ADD
 - Explanation:** ALTER TABLE table_name ADD column_name datatype adds a new column to an existing table.
- Answer:** a - font-weight: bold
 - Explanation:** font-weight: bold makes text bold. There is no text-style: bold or text-weight: bold in CSS.
- Answer:** b - Handles exceptions gracefully
 - Explanation:** try-except catches and handles exceptions, preventing program crashes.
- Answer:** b - <a>
 - Explanation:** The <a> tag creates hyperlinks with href attribute specifying the URL.
- Answer:** d - Both B and C
 - Explanation:** Both NOW() and CURRENT_TIMESTAMP return the current date and time.
- Answer:** b - World
 - Explanation:** "Hello World".split() splits by whitespace into ["Hello", "World"]. Index 1 is "World".
- Answer:** b - visibility: hidden
 - Explanation:** visibility: hidden hides the element but preserves its space. display: none removes it from flow.
- Answer:** a - raise
 - Explanation:** Python uses raise keyword to re-throw caught exceptions. throw is Java/C++.
- Answer:** b - DESCRIBE table_name
 - Explanation:** DESCRIBE (or DESC) shows the structure of a table including columns, types, and constraints.

Part B: Short Answer (8 x 2 = 16 marks)

1. CSV Reading Program

- Answer:**

```
import csv

with open('students.csv', 'r') as f:
    reader = csv.DictReader(f)
    students = list(reader)
```

```
# Sort by marks descending
students.sort(key=lambda x: int(x['Marks']), reverse=True)

# Display top 3
print("Top 3 Scorers:")
for i, s in enumerate(students[:3], 1):
    print(f"{i}. {s['Name']} - Roll: {s['Roll']} - Marks: {s['Marks']}")
```

- **Marking:** 1 mark for CSV reading, 1 mark for sorting and display.

2. JSON Output

- **Answer:**

```
{
  "students": [
    {
      "name": "A",
      "marks": 85
    },
    {
      "name": "B",
      "marks": 92
    }
  ]
}
```

`json.dumps()` converts dict to formatted JSON string. `json.loads()` parses JSON back to dict. Indexing accesses nested values.

- **Marking:** 1 mark for output, 1 mark for explanation.

3. Online Shopping Queries

- **Answer:**

```
-- (a) Customers from Bangalore
SELECT * FROM Customers WHERE city = 'Bangalore';

-- (b) Products with stock less than 10
SELECT * FROM Products WHERE stock < 10;

-- (c) Total orders by each customer
SELECT cid, COUNT(*) AS total_orders FROM Orders GROUP BY cid;

-- (d) Most ordered product
SELECT pid, SUM(quantity) AS total_qty FROM Orders GROUP BY pid ORDER BY total_qty
DESC LIMIT 1;
```

- **Marking:** 0.5 marks for each query.

4. List vs Tuple

- **Answer:** List: mutable, [], square brackets, slower, dynamic. Tuple: immutable, (), parentheses, faster, fixed. Use lists when data changes; use tuples for fixed data or dictionary keys.
- **Marking:** 1 mark for differences, 1 mark for use cases.

5. Pricing Card CSS

- **Answer:**

```
.card { box-shadow: 0 4px 8px rgba(0,0,0,0.1); border-radius: 10px; }
.card-header { background: linear-gradient(135deg, #667eea, #764ba2); color: white; }
```

```
.price { font-size: 2.5em; font-weight: bold; }
.features li { list-style: none; }
.features li::before { content: "✓ "; color: green; }
.cta-btn { background: #667eea; color: white; border: none; padding: 10px 20px; }
.cta-btn:hover { background: #764ba2; }
```

- **Marking:** 1 mark for card structure, 1 mark for styling.

6. Normalization

- **Answer:** Normalization organizes data to reduce redundancy. 1NF: atomic values, no repeating groups. 2NF: 1NF + no partial dependencies. 3NF: 2NF + no transitive dependencies. Example violation: 1NF - phone numbers in comma-separated string; 2NF - student name depends only on roll, not subject; 3NF - department name depends on department_id, not roll.
- **Marking:** 1 mark for definition, 1 mark for NF differences.

7. Merge Sort

- **Answer:**

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

- **Marking:** 1 mark for split logic, 1 mark for merge logic.

8. SQL JOINS

- **Answer:** JOIN combines rows from multiple tables. INNER JOIN: only matching rows. LEFT JOIN: all rows from left table, matching from right. RIGHT JOIN: all rows from right table, matching from left.

```
SELECT Students.name, Marks.marks
FROM Students
LEFT JOIN Marks ON Students.roll = Marks.roll;
```

- **Marking:** 1 mark for JOIN types, 1 mark for example.

Part C: Long Answer (4 x 3 = 12 marks)

1. Library Management System

- **Answer:** Complete Python program with: file handling for book records, functions for add_book, search_book, issue_book, return_book, display_all, try-except for file operations, daily report generation. Key elements: proper file modes, dictionary storage, input validation.
- **Marking:** 1 mark for file handling, 1 mark for functions, 1 mark for exception handling.

2. Healthcare Appointment System

- **Answer:** Complete HTML/CSS page with: header with hospital name and nav, appointment form with doctor selection and validation, styled table showing doctors with specializations, responsive media queries, professional medical colour scheme (blues/greens).
- **Marking:** 1 mark for structure, 1 mark for form/table, 1 mark for styling.

3. School Examination System Queries

- **Answer:**

```
-- (a) Result card
SELECT s.name, sub.sub_name, m.marks_obtained,
       SUM(m.marks_obtained) OVER(PARTITION BY s.roll) AS total,
       (SUM(m.marks_obtained) OVER(PARTITION BY s.roll) / (SELECT SUM(max_marks) FROM
Subjects)) * 100 AS percentage
FROM Students s JOIN Marks m ON s.roll = m.roll JOIN Subjects sub ON m.sub_id =
sub.sub_id;

-- (b) Class topper
SELECT name FROM Students WHERE roll = (SELECT roll FROM Marks GROUP BY roll ORDER BY
SUM(marks_obtained) DESC LIMIT 1);

-- (c) Subject-wise toppers
SELECT sub_name, name FROM Subjects sub JOIN Marks m ON sub.sub_id = m.sub_id JOIN
Students s ON m.roll = s.roll
WHERE m.marks_obtained = (SELECT MAX(marks_obtained) FROM Marks WHERE sub_id =
sub.sub_id);

-- (d) Failed students (marks < 35% of max)
SELECT DISTINCT name FROM Students WHERE roll IN (SELECT roll FROM Marks m JOIN
Subjects sub ON m.sub_id = sub.sub_id WHERE m.marks_obtained < sub.max_marks * 0.35);

-- (e) Overall pass percentage
SELECT (COUNT(DISTINCT CASE WHEN marks_obtained >= 35 THEN roll END) / COUNT(DISTINCT
roll)) * 100 FROM Marks;

-- (f) Rank list
SELECT RANK() OVER(ORDER BY SUM(marks_obtained) DESC) AS rank, name,
SUM(marks_obtained) AS total
FROM Students JOIN Marks ON Students.roll = Marks.roll GROUP BY roll ORDER BY total
DESC;
```

- **Marking:** 0.5 marks for each query.

4. Data Communication Process

- **Answer:** Components: Sender (source device), Receiver (destination), Medium (cable/wireless), Protocol (rules). Transmission types: Simplex (one-way), Duplex (two-way), Multiplex (multiple signals). Error detection: Parity check (counts bits), CRC (polynomial division). TCP role: segments data, adds sequence numbers, retransmits lost packets, uses checksums for integrity.
- **Marking:** 1 mark for components, 1 mark for transmission types, 1 mark for error detection and TCP.

Part D: Application Based (2 x 1 = 2 marks)

1. File Handling Fix

- **Answer:** Issue: No exception handling for missing file, manual close. Fix:

```
try:
    with open("data.txt") as f:
        content = f.read()
        print(content)
except FileNotFoundError:
    print("File not found!")
```

with is preferred because it automatically closes the file even if exceptions occur.

- **Marking:** 1 mark for try-except, 1 mark for with statement explanation.

2. Section-wise Query

- **Answer:**

```
SELECT section, AVG(marks) AS avg_marks,
       (SELECT name FROM Students s2 WHERE s2.section = s1.section ORDER BY marks
        DESC LIMIT 1) AS topper
FROM Students s1
GROUP BY section;
```

- **Marking:** 1 mark for correct query, 1 mark for clause explanation.